

아무도 적지 않은 명세

김민성, KAIST 전산학부

인공지능(AI)은 빠르게 코드를 생성하지만, 코드 생성의 속도가 보안을 보장하지는 않는다. 기능 요구사항은 프로그램이 해야 할 일을 몇 문장으로 적을 수 있는 반면, 보안 요구사항은 일어나서는 안 될 일을 다루므로 목록의 끝이 정해져 있지 않다. 이 글은 취약점(vulnerability)을 코드에 처음부터 보이는 오류라기보다, 뒤늦게 드러난 ‘아무도 적지 않았던 명세(specification)’로 본다. 그렇다고 모든 위험을 하나씩 나열할 필요는 없다. 메모리 안전성(memory safety)처럼 공격의 부류를 한 번에 덮는 명세를 적을 수 있고, 그런 명세는 테스트로는 확인할 수 없어 프로그램 검증(program verification)이 필요하다. 인공지능 시대에 우리가 할 일은 인공지능보다 빨리 코드를 읽고 쓰는 것이 아니라, 무엇이 일어나서는 안 되는지를 계속 찾아 적는 것이다.

2026 Security@KAIST의 주제는 ‘인공지능은 보안이다’였다. 다소 낯선 문장으로 들렸다. 인공지능은 코드를 작성하는 도구이기도 하지만 동시에 프로그램의 취약점을 찾을 수 있는 도구이기도 하다. 그렇다면 인공지능은 보안을 강화하는 편인가, 아니면 더 많은 보안 문제를 야기하는 편인가? 행사에 참가해 보니 이 질문은 인공지능의 능력 자체보다 우리가 인공지능에게 무엇을 요구하는지에 달려 있었다.

정렬 알고리즘의 명세는 두 줄이면 끝난다. ‘출력이 입력 원소들을 그대로 포함해야 하고, 정해진 순서대로 배열되어야 한다’는 조건을 식으로 옮기면 된다. 기능 요구사항은 프로그램이 무엇을 해야 하는지를 의미하므로 몇 문장이면 충분하다. 구현이 아무리 복잡해져도 적어야 할 목표는 더 늘지 않는다.

보안은 전혀 다른 이야기이다. 비밀번호가 노출되면 안 되고, 권한이 없는 사용자는 데이터에 접근할 수 없어야 하며, 입력 하나가 프로그램 전체를 멈추게 해서는 안 된다. 하나를 막아도 다른 가능성이 존재한다. 예상하지 못한 입력, 새로 연결된 외부 서비스, 만든 사람이 생각하지 못한 사용 방식이 계속 등장한다. 보안 요구사항은 ‘해야 할 일’이 아니라 ‘일어나서는 안 될 일’을 다루므로 우리가 적어야 하는 명세 목록의 끝이 정해져 있지 않다.

취약점은 암묵적이던 금지 조건이 뒤늦게 명세가 되는 순간 드러난다. 물론 버그는 코드에 존재한다. 그러나 입력 검증을 빠뜨린 코드가 곧바로 취약점인 것은 아니다. 그 입력이 비밀 정보를 꺼내거나 권한을 우회할 수 있고 따라서 그런 동작은 허용되어서는 안 된다는 조건이 분명해질 때 우리는 그것을 취약점이라 부른다. 취약점은 코드 안에 숨은 결함이면서 동시에 요구사항의 빈칸이다. 처음에는 아무도 그 요구사항을 쓰지 않았다.

코드 생성 능력이 아무리 높아져도 명세의 빈칸은 저절로 채워지지 않는다. 인공지능은 인간의 요구를 훌륭하게 만족시킬 수 있다. 하지만 ‘로그인 기능을 만들어 달라’는 요청만으로 실패 횟수 제한, 세션 탈취 방지, 권한 분리, 오류 메시지의 정보 노출 방지까지 보장되지는 않는다. 인공지능이 더 많은 코드를 더 빨리 만들수록 사람이 명시하지 않은 조건을 어길 기회도 함께 늘어난다.

프로그램 검증은 명세를 쓰고 그 명세가 지켜짐을 증명한다. 그렇다면 보안도 같은 방법으로 풀 수 있지 않을까? 일어나서는 안 될 일을 빠짐없이 명세로 적고 그것이 모두 지켜짐을 증명하면 된다. 실제로 검증은 명세 하나를 받아 그것이 모든 실행에서 성립하는지를 따진다. 문제는 그 목록을 누가 정하느냐다. 무엇이 위험인지는 프로그램 안에서 결정되지 않는다. 어제까지 아무 문제 없던 동작도 새로운 외부 서비스가 연결되거나, 지금까지는 알려지지 않았던 공격 기법이 등장하는 순간 취약점이 된다. 적어야 할 명세의 집합 자체가 시간과 함께 증가한다.

다행히도 개별 위험을 전부 나열할 수 없다는 사실이 보안을 포기할 이유가 되지 않는다는. 우리는 개별 공격을 끝없이 나열하는 대신 공격의 부류를 한 번에 막는 성질을 적을 수 있다. 대표적인 예가 바로 메모리 안전성이다. 어떤 입력이 버퍼를 넘칠지 하나하나 예측하지 않아도 ‘모든 메모리 접근이 허용된 범위 안에 있어야 한다’고 명세화할 수 있다. 명세 한 줄이 덮는 경우의 범위가 목록의 길이보다 중요하다.

높은 수준의 명세일수록 테스트로는 확인할 수 없다. 테스트는 선택한 입력에서 프로그램이 잘 동작함을 보여 주지만 아직 시도하지 않은 모든 입력에 대해서도 안전하다는 결론까지 주지는 못한다. 모든 메모리 접근이 범위 안에 있다는 주장은 입력을 아무리 많이 넣어 보아도 확인되지

않는다. 프로그램 검증은 그 간극을 논리로 메우려는 기술이다. 알려진 공격 하나를 막는 데서 그치지 않고 위험의 범위까지 문장 하나로 줄이려는 시도이기 때문이다. 그 위험에는 아직 이름도 붙지 않았다.

이번 학기에 짧은 프로그램의 사후조건(postcondition)을 작성할 때도 비슷한 어려움을 겪었다. 함수가 어떤 결과를 내야 하는지는 알고 있었다. 그러나 그 직관을 빠짐없는 조건으로 옮기려 하자 당연하다고 생각했던 전제들이 문장으로는 잘 잡히지 않았다. 무엇을 원하는지 몰라서가 아니라 이미 알고 있는 바를 명세로 표현하는 데에 애를 먹었던 것이다. 높은 수준의 명세를 적는 일은 여전히 사람의 몫이며, 인공지능이 코드를 대신 작성한다고 해서 그 어려움이 사라지지는 않는다.

인공지능이 코드 작성과 공격의 속도를 함께 높이는 시대일수록 우리는 안전하다는 말의 내용을 더 정교하게, 어쩌면 더 보수적으로 써야 한다. 보안은 빈칸 없는 프로그램에서 오지 않는다. 지금까지 보지 못한 빈칸을 발견할 때마다 그 자리에 아무도 적지 않았던 명세를 적는 것이다.